CORE JAVA / COLLECTIONS FRAMEWORK

Working with Set in Java

Student reference handout | 5 essential operations | 10 programming exercises

Learning goal: Create and manipulate sets confidently, understand the uniqueness rule, and choose between HashSet, LinkedHashSet, and TreeSet.

A **Set** stores unique elements. It does not provide index-based operations such as `get(0)` or `set(0, value)`.

HashSet	Unique elements; iteration order is not guaranteed.
LinkedHashSet	Unique elements; maintains insertion order.
TreeSet	Unique elements; maintains sorted order.

Use `import java.util.*;` and place the examples inside `main()`. The examples use Java 8-compatible features.

01 Adding elements

add(value) inserts one element; **addAll(collection)** adds multiple values. Duplicate additions do not change the set.

```
Set<Integer> a = new LinkedHashSet<>();
a.add(10);
a.add(20);
boolean inserted = a.add(10);  // false: duplicate

Set<Integer> b = new HashSet<>(Arrays.asList(20, 30, 40));
a.addAll(b);
System.out.println(a);         // [10, 20, 30, 40]
System.out.println(inserted);  // false
```

Remember: Set.add() returns true only if the Set changes. LinkedHashSet makes the output order predictable.

02 Removing elements

Use **remove(value)** to remove one value, **removeAll(collection)** to remove matches, **removeIf(condition)** for conditional deletion, and **clear()** to empty the set.

```
Set<Integer> s = new LinkedHashSet<>(Arrays.asList(10, 15, 20, 25, 30));
s.remove(15);                      // removes value 15
s.removeAll(Arrays.asList(20, 30));
s.removeIf(n -> n % 2 == 0);      // removes even numbers
System.out.println(s);             // [25]
s.clear();                         // s is now empty
```

Remember: Unlike List.remove(int), Set.remove(value) removes by value because Set has no indices.

OPERATIONS / PRACTICAL EXAMPLES

03 Updating elements

A Set has **no set(index, value)** method. To replace a value, remove the old one and add the new one. Check whether the old value exists when replacement should be conditional.

```
Set<String> names = new LinkedHashSet<>(
    Arrays.asList("Ravi", "Anu", "Kiran")
);

if (names.remove("Ravi")) {
    names.add("Rahul");
}
System.out.println(names);   // [Anu, Kiran, Rahul]
```

Remember: Re-inserting into LinkedHashSet places the new value at the end. If it already exists, a duplicate is not added.

04 Iterating over a Set

A Set cannot be traversed through numeric indices. Use an enhanced **for** loop, an **Iterator**, or **forEach()**.

A. Enhanced for loop

```
Set<Integer> s = new LinkedHashSet<>(Arrays.asList(10, 20, 30));
for (int n : s) {
    System.out.println(n);
}
```

B. Iterator

```
Iterator<Integer> itr = s.iterator();
while (itr.hasNext()) {
    System.out.println(itr.next());
}
```

C. forEach() with a lambda

```
s.forEach(n -> System.out.println(n));
```

D. Safely removing elements during iteration

```
Iterator<Integer> itr2 = s.iterator();
while (itr2.hasNext()) {
    if (itr2.next() > 20) {
        itr2.remove();
    }
}
System.out.println(s);  // [10, 20]
```

Remember: Removing directly from the Set inside an enhanced-for loop is unsafe. Use Iterator.remove() or removeIf() for safe conditional removal.

OPERATIONS / ORDERING & COMMON MISTAKES

05 Sorting a Set

TreeSet keeps values sorted by their natural ordering or a supplied **Comparator**. HashSet has no defined order; LinkedHashSet maintains insertion order, not sorted order.

A. Ascending order (TreeSet)

```
Set<Integer> s = new TreeSet<>();
s.addAll(Arrays.asList(40, 10, 30, 20));
System.out.println(s);   // [10, 20, 30, 40]
```

B. Descending order (Comparator)

```
Set<Integer> s = new TreeSet<>(Collections.reverseOrder());
s.addAll(Arrays.asList(40, 10, 30, 20));
System.out.println(s);   // [40, 30, 20, 10]
```

C. Sorting the values of an existing HashSet

```
Set<Integer> original = new HashSet<>(Arrays.asList(40, 10, 30, 20));
Set<Integer> sortedSet = new TreeSet<>(original);
System.out.println(sortedSet);    // [10, 20, 30, 40]

List<Integer> sortedList = new ArrayList<>(original);
Collections.sort(sortedList);
System.out.println(sortedList);   // [10, 20, 30, 40]
```

Remember: Collections.sort() works with List, not Set. Do not assume a HashSet or LinkedHashSet is sorted.

QUICK REFERENCE / METHODS BY OPERATION

Adding	add(value), addAll(collection)
Removing	remove(value), removeAll(collection), removeIf(predicate), clear()
Updating	remove(oldValue), then add(newValue)
Iterating	Enhanced for loop, iterator(), forEach()
Sorting	TreeSet, TreeSet(comparator), or convert to List and sort

Common mistakes to avoid

1. Trying to call `get(index)` or `set(index, value)` on a Set.
2. Expecting `HashSet` to print values in insertion or sorted order.
3. Trying to sort a Set using `Collections.sort(set)`.
4. Removing directly from a Set during enhanced-for iteration.

STUDENT PRACTICE / 10 PROGRAMMING EXERCISES

Write a Java program for each problem. Choose the appropriate Set implementation and justify your choice where relevant.

01. Store unique numbers. Read 10 integers, add them to a Set, and display only distinct values in the order of their first appearance.
Test data: 10, 20, 10, 30, 20, 40

02. Merge two sets. Create two Sets of integers and add all elements of the second into the first. Print the result and explain why duplicates are absent.
Test data: A = {10, 20, 30}, B = {20, 40, 50}

03. Remove a selected element. Read a number from the user, remove it from a Set, and print whether the element was successfully removed.
Test data: Set = {5, 10, 15, 20}; remove 15

04. Remove matching values. Given two Sets, remove from the first every value present in the second. Print the modified first Set.
Test data: A = {1, 2, 3, 4, 5}, B = {2, 4, 6}

05. Conditional removal. Use removeIf() to remove all odd integers from a Set and print the remaining elements.
Test data: Set = {11, 12, 13, 14, 15, 16}

06. Replace a name. Replace "Ravi" with "Rahul" in a LinkedHashSet. If "Ravi" is absent, do not insert "Rahul".
Test data: {Ravi, Anu, Kiran}

07. Sum using iteration. Traverse a Set using an enhanced for loop, and calculate the sum of all its elements.
Test data: {5, 10, 15, 20}

08. Safe Iterator removal. Use an Iterator to remove all numbers greater than 50 from a Set. Do not use removeIf().
Test data: {25, 70, 40, 90, 50, 60}

09. Sort ascending and descending. Store 10 numbers in a TreeSet and print both ascending and descending sequences.
Test data: 40, 10, 70, 20, 30, 50, 10, 60, 80, 90

10. Sort a HashSet without changing it. Create a HashSet of salaries and produce a separate descending sorted collection. Print both collections.
Test data: {45000, 30000, 60000, 50000}

Submission checklist: Use Set operations rather than List indices; verify duplicates; use safe iteration for removal; choose TreeSet or a sorted List for ordered results.